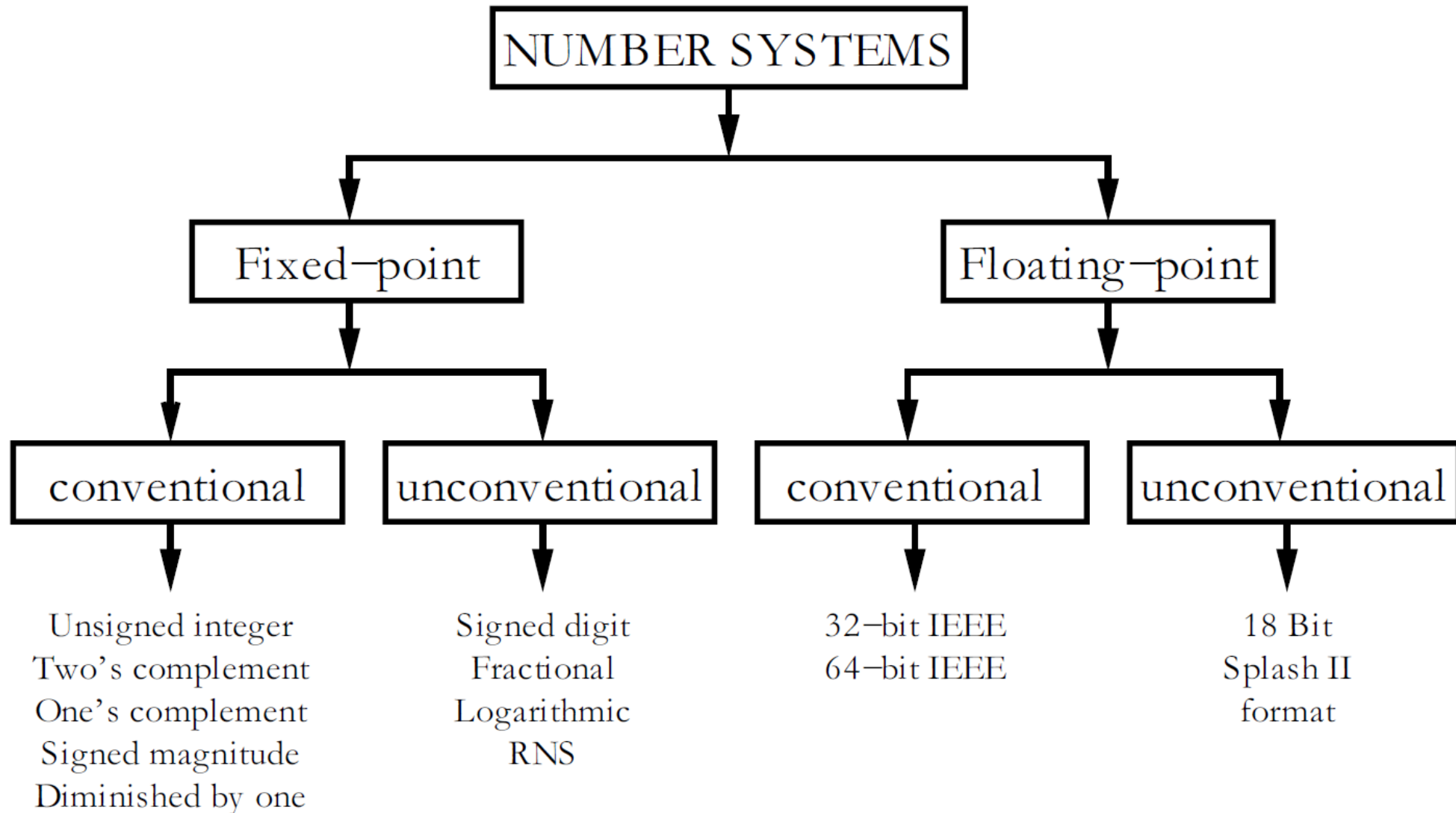




Lecture 2

Number Systems

How to Represent Number?



Number Systems



➤ Fixed point numbers

❖ Conventional

- ☐ Signed-magnitude
- ☐ One's complement
- ☐ Two's complement

➤ Floating point numbers

Decimal Number System



➤ Decimal number system uses the 10 symbols (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) to represent a number

➤ Example:

$$(456)_{10} = 4 \times 10^2 + 5 \times 10^1 + 6 \times 10^0$$

$$(3705.86)_{10} = 3 \times 10^3 + 7 \times 10^2 + 0 \times 10^1 + 5 \times 10^0 + 8 \times 10^{-1} + 6 \times 10^{-2}$$

Binary Number System

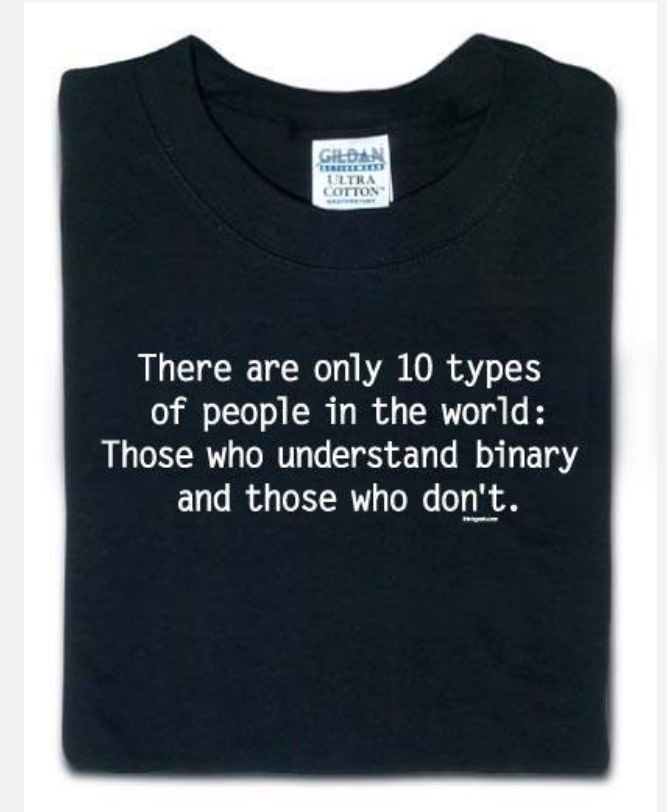


➤ In binary number system, 2 symbols (0 and 1) are used to represent a number

➤ Example:

$$\begin{array}{l} \begin{array}{c} \text{2}^4 \quad \text{2}^3 \quad \text{2}^2 \quad \text{2}^1 \quad \text{2}^0 \\ \nearrow \nearrow \nearrow \nearrow \nearrow \\ (11001)_2 \end{array} \\ \begin{aligned} &= (2^4)_{10} + (2^3)_{10} + (2^0)_{10} \\ &= (16)_{10} + (8)_{10} + (1)_{10} \\ &= (25)_{10} \end{aligned} \end{array}$$

$$\begin{aligned} (101.01)_2 &= (2^2)_{10} + (2^0)_{10} + (2^{-2})_{10} \\ &= (4)_{10} + (1)_{10} + (0.25)_{10} \\ &= (5.25)_{10} \end{aligned}$$



Binary Number System



➤ Unsigned binary

$$X = x_{W-1}x_{W-2}\dots x_0 = \sum_{k=0}^{W-1} x_k \cdot 2^k, \quad x_k \in \{0, 1\}$$

➤ The range of an N -bit unsigned binary number is $[0, 2^N-1]$

❖ The largest 4-bit number is $(1111)_2 = 16$

➤ Negative number is not represented. To represent negative numbers, an extra bit, called sign bit is needed

Fixed-point Number



- In VLSI implementation, fixed-point arithmetic is preferred over floating point
 - ❖ higher speed
 - ❖ Lower cost
- Represented by an integer with a scaling factor

$$X = x_{W-1}x_{W-2}\dots x_M . x_{M-1}\dots x_0 = x_{W-1}x_{W-2}\dots x_0 \times r^{-M}$$

Negative Numbers



- Three approaches to represent negative numbers
 - ❖ Sign and magnitude
 - ❖ One's-complement
 - ❖ Two's-complement

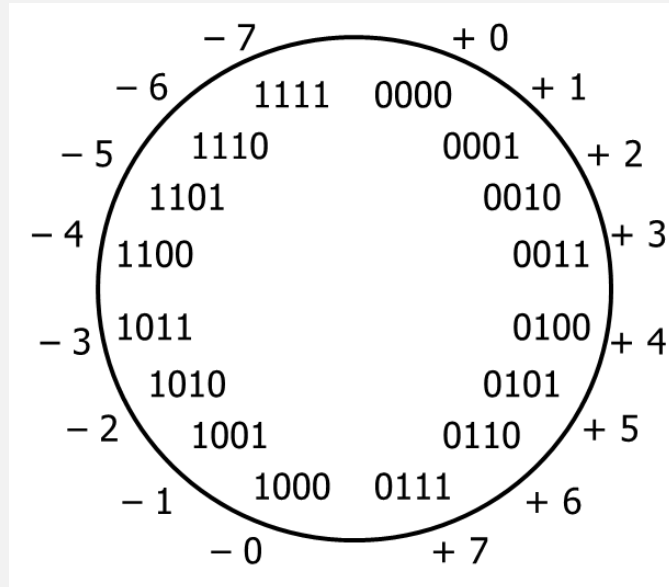
- All 3 approaches represent positive numbers in the same way

Signed-magnitude



- The most significant bit (MSB) is the sign bit
- Remaining bits are the number's magnitude

$$X = x_{W-1}x_{W-2}...x_0 = (-1)^{x_{W-1}} \sum_{k=0}^{W-2} x_k \cdot 2^k, \quad x_k \in \{0, 1\}.$$



Signed-magnitude

➤ Problem 1: Two representations of for zero

$$\diamond +0 = 0000 \text{ and } -0 = 1000$$

➤ Problem 2: Arithmetic is cumbersome

$$\diamond 4 - 3 \neq 4 + (-3)$$

Add		Subtract			Compare and subtract		
4	0100	4	0100	0100	- 4	1100	1100
+ 3	+ 0011	- 3	+ 1011	- 0011	+ 3	+ 0011	- 0011
= 7	= 0111	= 1	≠ 1111	= 0001	- 1	≠ 1111	= 1001

One's Complement

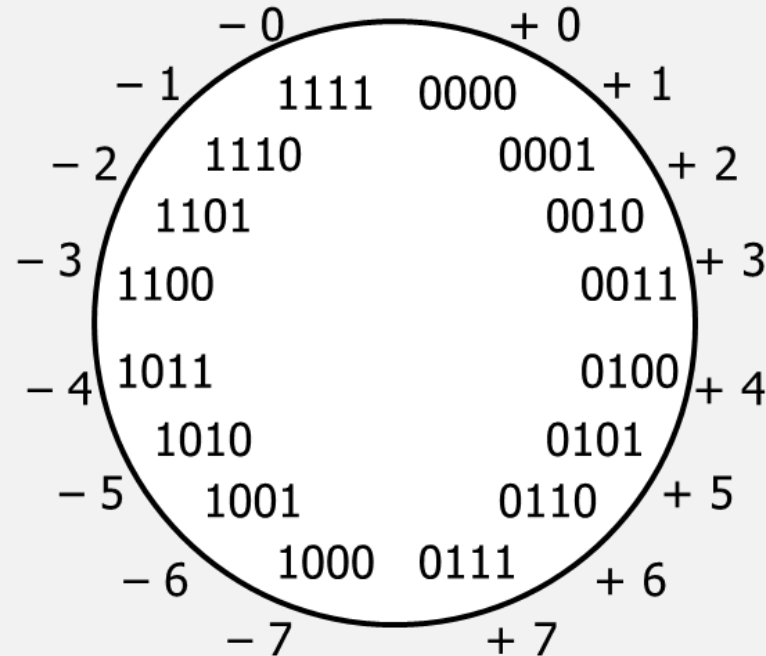


➤ Bitwise complement of positive number

$$\diamond 0111 \equiv 7_{10}$$

$$\diamond 1000 \equiv -7_{10}$$

➤ For a 4-bit one's complement number

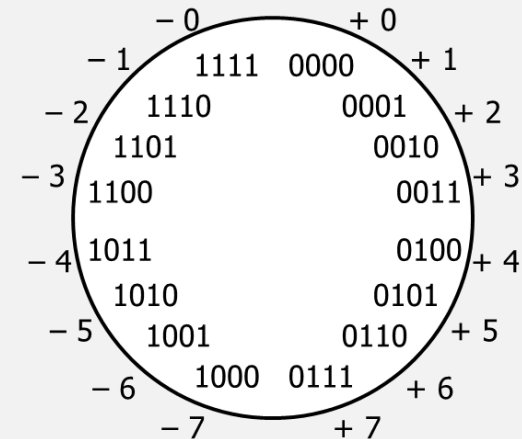


One's Complement

➤ Solves the arithmetic problem

Add		Invert, add, add carry		Invert and add	
4	0100	4	0100	- 4	1011
+ 3	+ 0011	- 3	+ 1100	+ 3	+ 0011
= 7	= 0111	= 1	1 0000	- 1	1110
		add carry:	+1		
			= 0001		

end-around carry



*Why One's Complement Works



➤ The one's complement of an 4-bit positive number y is $1111_2 - y$

$$\diamond 0111 \equiv 7_{10}$$

$$\diamond 1111_2 - 0111_2 = 1000_2 \equiv -7_{10}$$

➤ What is 1111_2 ?

$$\diamond 1 \text{ less than } 10000_2 = 2^4 - 1$$

$$\diamond -y \text{ is represented by } (2^4 - 1) - y$$

*Why One's Complement Works



- Adding x and $-y$ (x, y are positive numbers), we get $x + ((2^4 - 1) - y) = (2^4 - 1) + (x - y)$
 - ❖ If $x < y$, i.e., $x - y < 0$, there will be no carry from $(2^4 - 1) + (x - y)$. Just add representations to get correct negative number.
 - ❖ If $x > y$, i.e., $x - y > 0$, there will be a carry. Performing end-around carry subtracts 2^4 and adds 1, subtracting $(2^4 - 1)$ from $(2^4 - 1) + (x - y)$
 - ❖ If $x = y$, then answer should be 0, get $(2^4 - 1) = 1111_2$
- Still have two representations for zero!
 - ❖ $+0 = 0000$ and also $-0 = 1111$

Two's Complete



➤ Negative number: one's complement **plus one**

$$\diamond 0111 \equiv 7_{10}$$

$$\diamond 100\mathbf{1} \equiv -7_{10}$$

➤ The value of a two's complement number is

$$X = x_{W-1}x_{W-2}\dots x_0 = -x_{W-1} \cdot 2^{W-1} + \sum_{k=0}^{W-2} x_k \cdot 2^k, \quad x_k \in \{0, 1\}.$$

➤ The MSB carries a negative weight

$$(1101)_{2's} = -2^3 + 2^2 + 2^0 = -8 + 4 + 1 = -3$$

$$(1001)_{2's} = -2^3 + 2^0 = -8 + 1 = -7$$

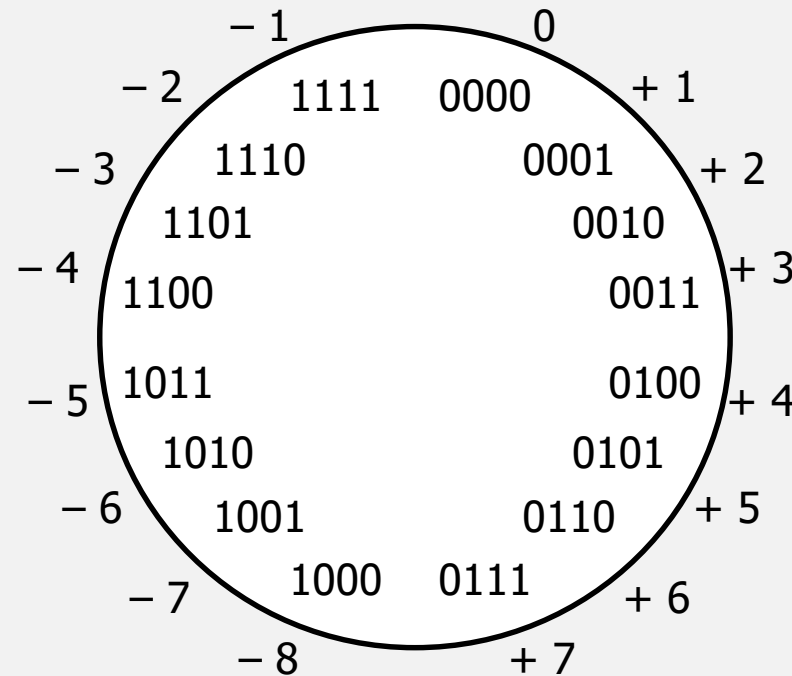
$$(0110)_{2's} = 2^2 + 2^1 = 4 + 2 = 6$$

$$(110)_{2's} = -2^2 + 2^1 = -4 + 2 = -2$$

Two's Complete



- The range of an N -bit two's complement number is $[-2^{N-1}, 2^{N-1}-1]$
- For a 4-bit two's complement number



Two's Complete



➤ Benefits:

- ❖ Simplified arithmetic
- ❖ Only one zero!

Add		Invert and add		Invert and add	
4	0100	4	0100	- 4	1100
+ 3	+ 0011	- 3	+ 1101	+ 3	+ 0011
= 7	= 0111	= 1	1 0001	- 1	1111
		drop carry	= 0001		

➤ As long as the results can be represented (no overflow)!

*Why Two's Complete Works

- Recall: The one's complement of a b -bit positive number y is $(2^b - 1) - y$
- Two's-complement adds 1 to one's complement, thus, $-y$ is $2^b - y$
 - ❖ $(-y) \% 2^b = (2^b - y) \% 2^b$ (have the same remainder when divided by 2^b)
 - ❖ Ignoring carries is equivalent to doing $\% 2^b$

*Why Two's Complete Works

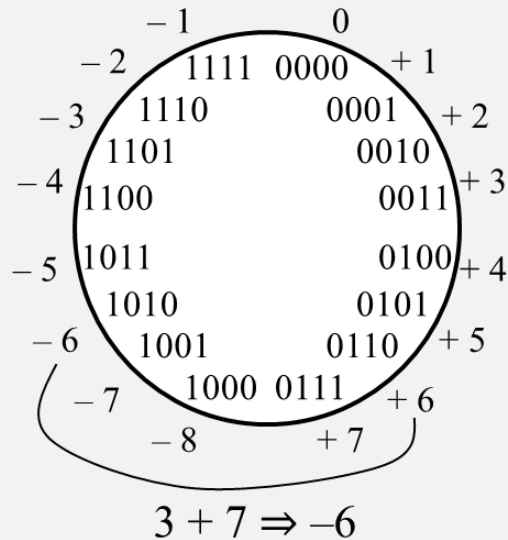


- Adding x and $-y$ (x, y are positive numbers) we get $x + (2^b - y) = 2^b + (x - y)$
 - ❖ If there is a carry, that means that $x \geq y$. Dropping the carry yields $x - y$
 - ❖ If there is no carry, i.e., $x < y$, then we can think of it as $2^b - (y - x)$, which is the 2's complement of $x - y$
- The results are correct as long as the results can be represented (no overflow)!

Overflow



- The sum of 2 positive numbers can be negative
- The sum of 2 negative numbers can be positive



$$\begin{array}{r} 3 \\ + 7 \\ \hline 10 \end{array} \qquad \begin{array}{r} 0011 \\ + 0111 \\ \hline 1010 \\ \underbrace{\hspace{1.5cm}}_{-6 \text{ Negative !}} \end{array}$$

- This is because a 4-bit two's complement number has a range of $[-2^3, 2^3-1]$, i.e., $[-8, 7]$, it cannot represent 10. **Overflow has occurred!**

Overflow



- Overflow may occur during an add operation when operands are both positive or negative.
- Consider the following add operation

$$\begin{array}{rcccccccc} & a_{N-1} & a_{N-2} & \times & \times & \times & a_1 & a_0 \\ + & b_{N-1} & b_{N-2} & \times & \times & \times & b_1 & b_0 \\ \hline s_{N-1} & s_{N-2} & \times & \times & \times & s_1 & s_0 \end{array}$$

Positive + Positive



0 X X X X

+ 0 X X X X

 No carry-out from MSB when adding 2 positive numbers, since MSB summation is $0 + 0 + \text{carry into MSB}$.

0 X X X X

+ 0 X X X X

 Whether there is carry into MSB depends on the values of Xs.

Positive + Positive

$$\begin{array}{r} 0 \ 1 \ 0 \ 1 \ 1 = (11)_{10} \\ + 0 \ 1 \ 0 \ 0 \ 1 = (9)_{10} \\ \hline 1 \ 0 \ 1 \ 0 \ 0 = (-12)_{10} \end{array}$$

There is carry into MSB.

$$\begin{array}{r} 0 \ 1 \ 0 \ 1 \ 1 = (11)_{10} \\ + 0 \ 0 \ 1 \ 0 \ 1 = (5)_{10} \\ \hline 1 \ 0 \ 0 \ 0 \ 0 = (-16)_{10} \end{array}$$

There is carry into MSB.

- The result is wrong if there is carry into the MSB because it makes the sum negative

Positive + Positive

$$\begin{array}{r} 0 \ 0 \ 0 \ 1 \ 1 = (3)_{10} \\ + 0 \ 1 \ 0 \ 0 \ 1 = (9)_{10} \\ \hline 0 \ 1 \ 1 \ 0 \ 0 = (12)_{10} \end{array}$$

There is no carry into MSB.

$$\begin{array}{r} 0 \ 0 \ 0 \ 1 \ 1 = (3)_{10} \\ + 0 \ 0 \ 1 \ 0 \ 1 = (5)_{10} \\ \hline 0 \ 1 \ 0 \ 0 \ 0 = (8)_{10} \end{array}$$

There is no carry into MSB.

➤ The result is correct if there is no carry into MSB

Negative + Negative



1 X X X X

+ 1 X X X X



There is carry-out from MSB when adding 2 negative numbers, since MSB summation is $1 + 1 + \text{carry into MSB}$.

1 X X X X

+ 1 X X X X



Whether there is carry into MSB depends on the values of Xs.

Negative + Negative



$$\begin{array}{r} 1\ 0\ 0\ 1\ 1 = (-13)_{10} \\ +\ 1\ 1\ 0\ 0\ 1 = (-7)_{10} \\ \hline 0\ 1\ 1\ 0\ 0 = (12)_{10} \end{array}$$

There is no carry into MSB.

$$\begin{array}{r} 1\ 0\ 0\ 1\ 1 = (-13)_{10} \\ +\ 1\ 0\ 1\ 0\ 1 = (-11)_{10} \\ \hline 0\ 1\ 0\ 0\ 0 = (8)_{10} \end{array}$$

There is no carry into MSB.

- The result is wrong if there is no carry into the MSB because it makes the sum positive

Negative + Negative



$$\begin{array}{r} 1 \ 1 \ 0 \ 1 \ 1 = (-5)_{10} \\ + 1 \ 1 \ 0 \ 0 \ 1 = (-7)_{10} \\ \hline 1 \ 0 \ 1 \ 0 \ 0 = (-12)_{10} \end{array}$$

There is carry into MSB.

$$\begin{array}{r} 1 \ 1 \ 0 \ 1 \ 1 = (-5)_{10} \\ + 1 \ 0 \ 1 \ 0 \ 1 = (-11)_{10} \\ \hline 1 \ 0 \ 0 \ 0 \ 0 = (-16)_{10} \end{array}$$

There is carry into MSB.

➤ The result is correct if there is carry into MSB

Positive + Negative ?



$$\begin{array}{r} 0 \ X \ X \ X \ X \\ + \ 1 \ X \ X \ X \ X \\ \hline \end{array}$$

There is carry-out from MSB if there is carry into MSB because MSB addition is $0 + 1 + \text{carry in to MSB}$.

➤ The result is always correct

❖ Let $S=A+B$. If A is positive and B is negative. Then $B < S < A$. The word-length is sufficient to represent S if it is sufficient to represent A

*Overflow Detection

➤ Adding two positive numbers

- ❖ There is no carry-out from the MSB. Overflow occurs if there is carry-in to the MSB

➤ Adding two negative numbers

- ❖ There is carry-out from the MSB. Overflow occurs if there is no carry-in to the MSB

➤ Adding a positive number and a negative number

- ❖ Result is always correct

➤ To summarize

- ❖ Overflow occurs if and only if carry-in to the MSB $\neq$ carry-out from the MSB

Sign Extension

- In two's complement addition, input and output operands must be the same word-length because of implied zeros

$$\begin{array}{r} 001011 \quad -5 \\ 010010 \quad +18 \\ \hline 011101 \quad +29! \end{array}$$

- Sign extension is needed!

Sign Extension



➤ Prefix numbers with the MSB, e.g.,

$$(101)_{2's} = -4 + 1 = -3$$

$$(1101)_{2's} = -8 + 4 + 1 = -3$$

$$(11101)_{2's} = -16 + 8 + 4 + 1 = -3$$

$$(111101)_{2's} = -32 + 16 + 8 + 4 + 1 = -3$$

$$\therefore (101)_{2's} = (1101)_{2's} = (11101)_{2's} = (1\cdots 101)_{2's}$$

$$(0101)_{2's} = 4 + 1 = 5$$

$$(00101)_{2's} = 4 + 1 = 5$$

$$\therefore (0101)_{2's} = (00101)_{2's} = (000101)_{2's} = (0\cdots 0101)_{2's}$$

➤ Preserve the sign and value of the number!

Sign Extension

➤ Needed for two's complement addition

❖ Consider adding two numbers of different widths

$$\begin{array}{r} 1011-5 \\ 010010+18 \\ \hline 011101+29! \end{array}$$

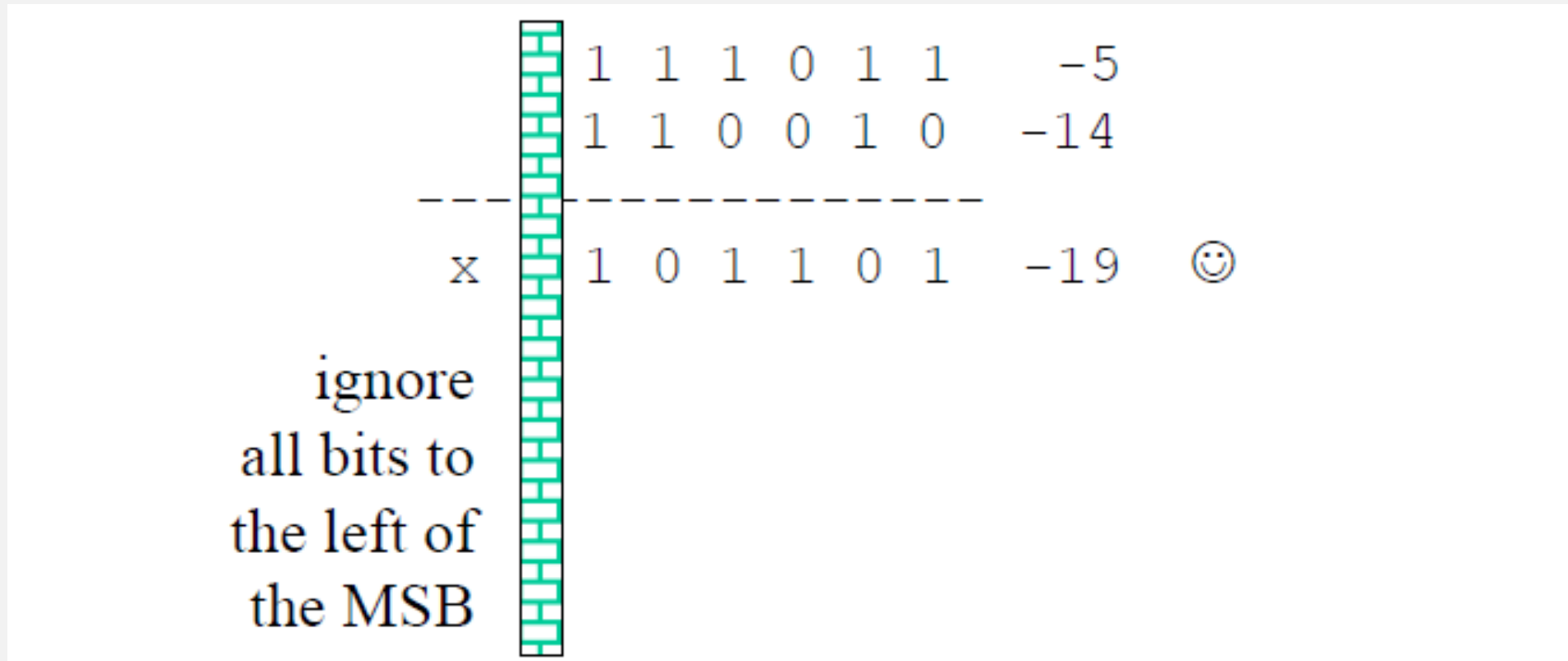
❖ If we do a sign extension for -5

$$\begin{array}{r|l} 111011 & -5 \\ 010010 & +18 \\ \hline x001101 & +13 \quad \text{😊} \end{array}$$

Sign Extension

➤ Ignore carry-out bits (mod 2^b)

❖ Do not spend any hardware calculating any bits to the left of the answer's MSB



Sign Extension



- In two's complement addition, the result is correct as long as it is in the range of $[-2^{N-1}, 2^{N-1}-1]$

$$\begin{array}{r} 3 \\ + 7 \\ \hline 10 \\ + (-5) \\ \hline 5 \end{array}$$

$$\begin{array}{r} 0011 \\ + 0111 \\ \hline 1010 \rightarrow -6 \text{ Negative!} \\ + 1011 \\ \hline 10101 \end{array}$$

5 $\Rightarrow$ correct result

This is because the operands can be sign extended.

$$\begin{array}{r} 3 \\ + 7 \\ \hline 10 \\ + (-5) \\ \hline 5 \end{array}$$

$$\begin{array}{r} 00011 \\ + 00111 \\ \hline 01010 \rightarrow 10 \text{ (correct result)} \\ + 11011 \\ \hline 00101 \end{array}$$

5 $\Rightarrow$ correct result

Virtual sign extension

Number Systems



➤ Fixed point numbers

❖ Conventional

- ☐ Signed-magnitude
- ☐ One's complement
- ☐ Two's complement

➤ Floating point numbers

Floating-point Number



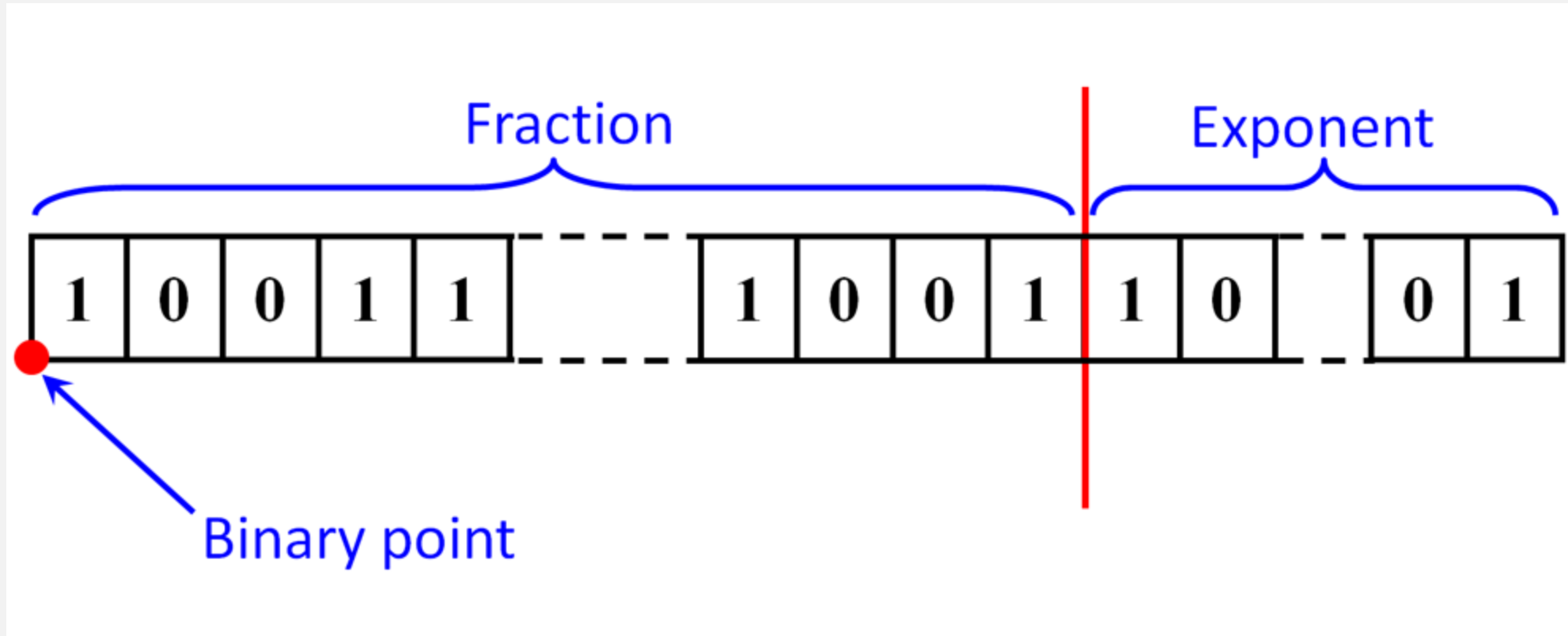
$$A = P \times Q^D$$

Diagram illustrating the components of a floating-point number representation:

- P is labeled "fraction; mantissa" (indicated by a blue arrow).
- Q is labeled "base; radix" (indicated by a blue arrow).
- D is labeled "exponent; characteristic" (indicated by a blue arrow).

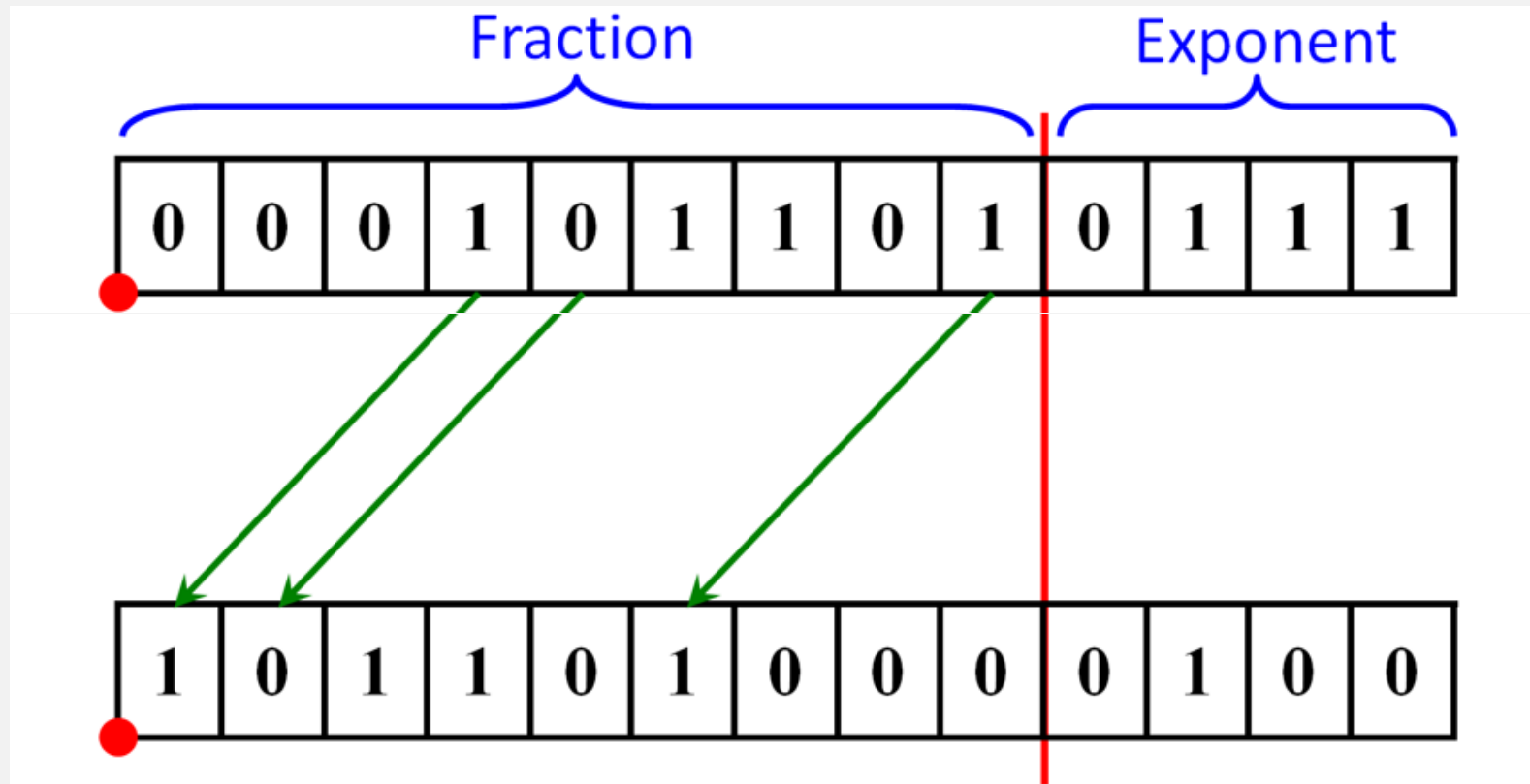
Example: 8934 can be written as 0.8934×10^4

Binary Representation of Floating-point



Binary Representation of Floating-point

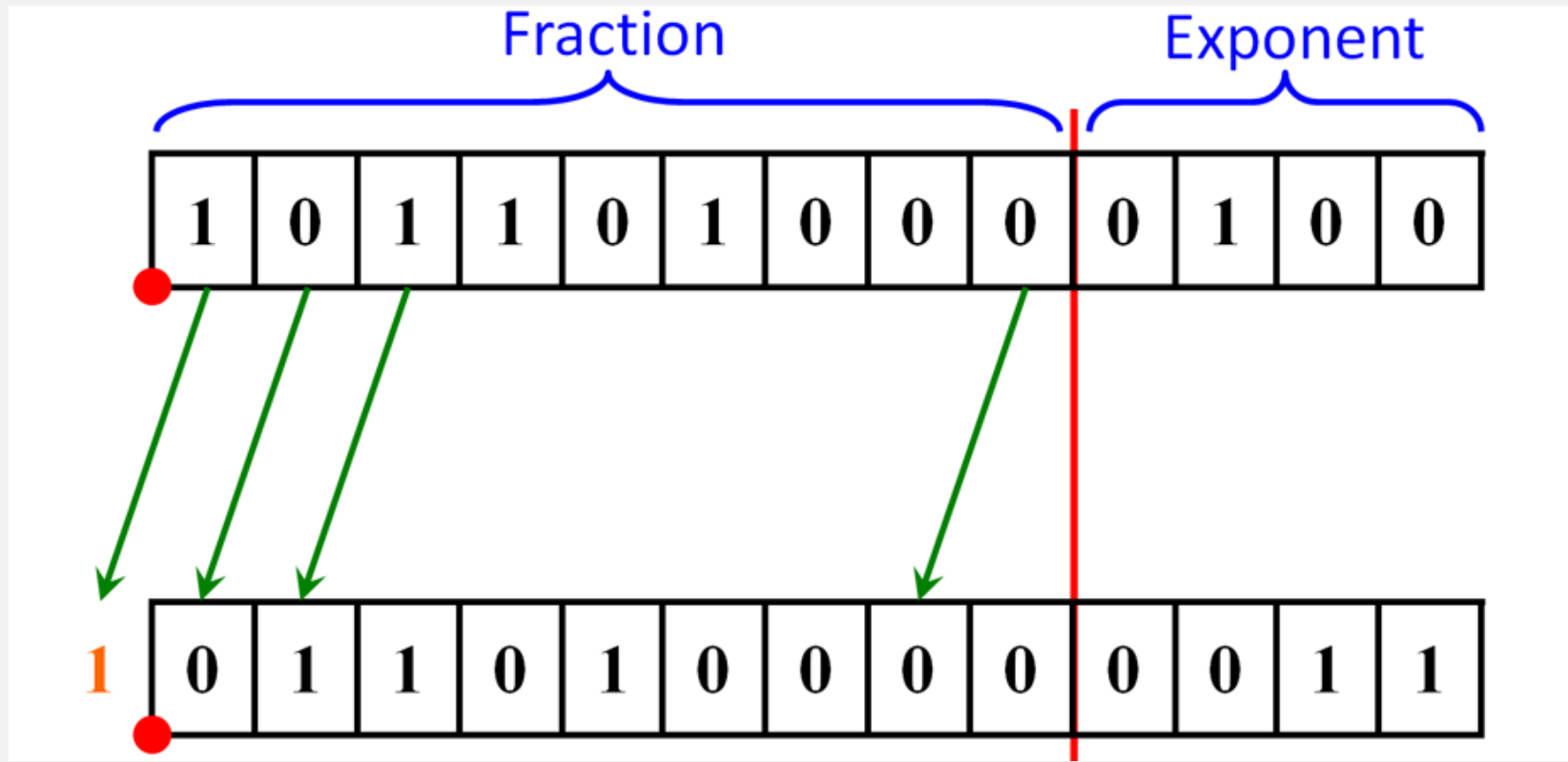
- For maximum precision, the number can be normalized until the first digit is “1”



Binary Representation of Floating-point



➤ Since the first digit is a “1”, it is not necessary to record it

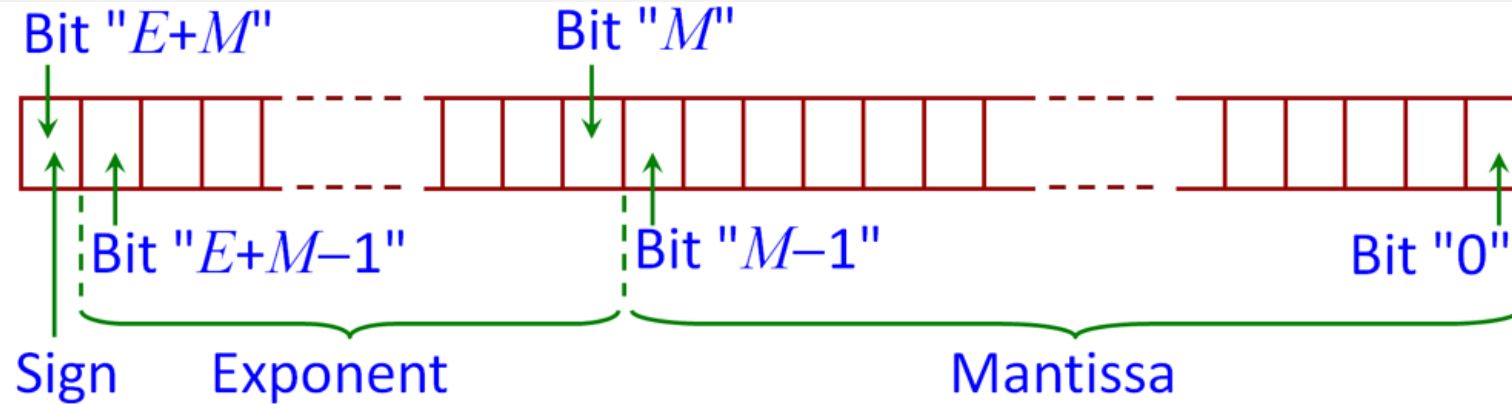


IEEE 754 Number Format



- IEEE standard for binary floating-point arithmetic
- IEEE 754 specifies 4 formats
 - ❖ Single-precision (32-bit)
 - ❖ Double-precision (64-bit)
 - ❖ Signal-extended precision (≥ 43 -bit, seldom used)
 - ❖ Double-extended precision (≥ 79 -bit, usually 80-bit)

IEEE 754 Number Format



	Exponent	Mantissa
NaNs	$2^E - 1$, i.e. all 1s	non zero
Infinities	$2^E - 1$, i.e. all 1s	0
Zeroes	0, i.e. all 0s	0
Denormalized numbers	0, i.e. all 0s	non zero
Normalized numbers	1 to $2^E - 2$. Biased binary	Any number

IEEE 754 Number Format



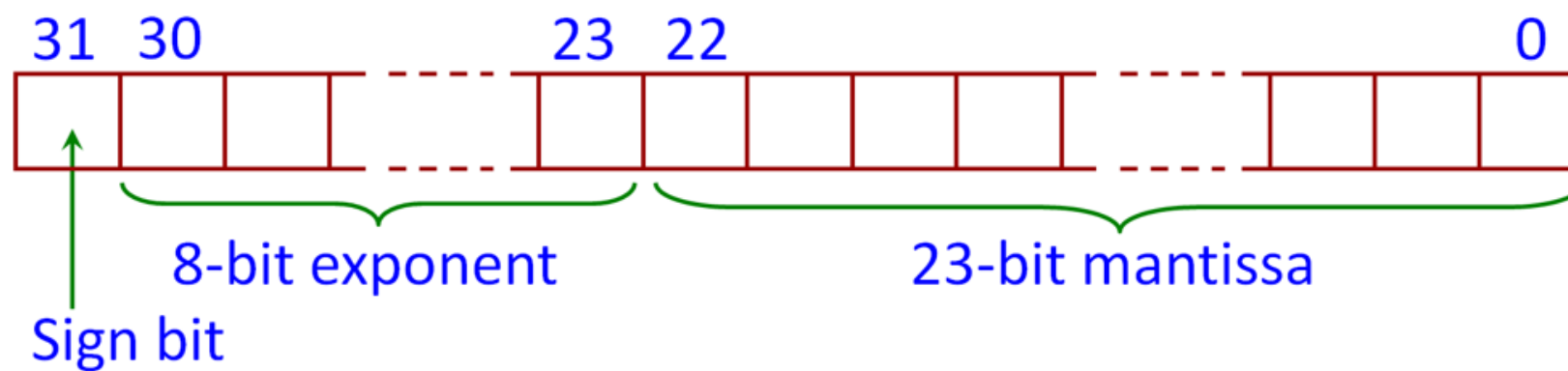
➤ Sign bit

- ❖ Number is positive if sign bit is “0”
- ❖ Number is negative if sign bit is “1”

➤ Biased exponent

- ❖ The exponent is a signed value
 - for large and small magnitudes
- ❖ Two's complement is **not** used
- ❖ A constant $2^{E-1}-1$ is added to the exponent
 - E.g., for $E=8$, the exponent bias is $2^7-1=127$, if the exponent is -3 , it will be recorded as $-3+127=124$, i.e., $(01111100)_2$

32-bit Single Precision Format



$$\text{Value} = (-1)^S \times 2^{\text{Exp}-127} \times M$$

Where

Exp = Recorded exponent.

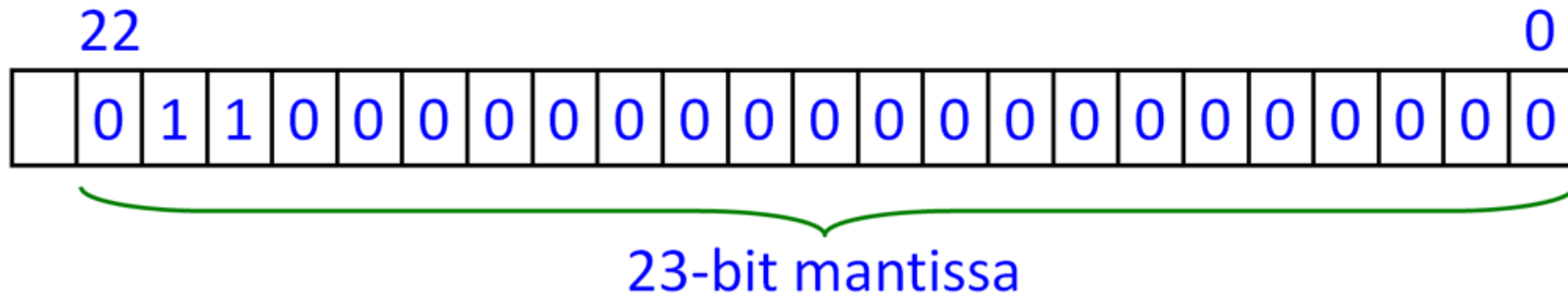
M = 1.(value represented by fractional bit).

The Mantissa Value



Mantissa value = 1.(value represented by fractional bit).

Example



$$\begin{aligned}\text{Mantissa value} &= 1 + 2^{-2} + 2^{-3} \\ &= 1 + 0.25 + 0.125 \\ &= 1.375\end{aligned}$$

Fixed-point vs. Floating-point



➤ Example: 32bit

➤ For fixed point

The smallest 1×2^{-N}

The largest $(2^{32} - 1) \times 2^{-N}$

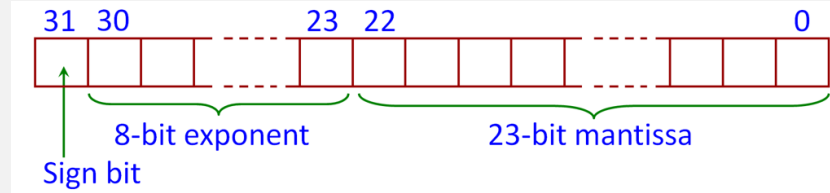
Dynamic range $20\log\left(\frac{(2^{32} - 1) \times 2^{-N}}{1 \times 2^{-N}}\right) \approx 192\text{dB}$

➤ For floating point

The smallest $1.000000000000000000000000 \times 2^{-126} \approx 1.175494 \times 10^{-38}$

The largest $1.111111111111111111111110 * 2^{128} \approx 3.402823 \times 10^{38}$

Dynamic range $20\log\left(\frac{3.402823 \times 10^{38}}{1.175494 \times 10^{-38}}\right) \approx 1667.6\text{dB}$



$$\text{Value} = (-1)^S \times 2^{\text{Exp}-127} \times M$$

Where

Exp = Recorded exponent.

M = 1.(value represented by fractional bit).

Fixed-point vs. Floating-point



- Fixed point
 - ❖ Limited dynamic range, fast, low-power
- Floating point
 - ❖ High dynamic range, complex, slow
- Example
 - ❖ Filter coefficient quantization